

1. Perhatikan potongan program berikut:

```
int total = 0;
for (int i = 1; i <= N; i++) {
    for (int j = N - 3; j >= i; j--) {
        total = total + 1;
    }
}
```

Kompleksitas potongan program tersebut paling mendekati....

- ☐ a  $O(\log N)$
- ☒ b  $O(N^2)$
- ☐ c  $O(N^3)$
- ☐ d  $O(N - 3)$

## Pembahasan

Perhatikan kode berikut:

⟨⟩ C++

```
int total = 0;

for (int i = 1; i <= N; i++) {
    for (int j = N - 3; j >= i; j--) {
        total = total + 1;
    }
}
```

### 1. Analisis Perulangan Luar

⟨⟩ C++

```
for (int i = 1; i <= N; i++)
```

Perulangan luar berjalan sebanyak  $N$  kali.

Kompleksitas:

$O(N)$

## 2. Analisis Perulangan Dalam

↗ C++

```
for (int j = N - 3; j >= i; j--)
```

Jumlah iterasi bergantung pada nilai `i`.

- Saat `i = 1` → sekitar  $N - 3$  kali
- Saat `i = 2` → sekitar  $N - 4$  kali
- Saat `i = 3` → sekitar  $N - 5$  kali
- ...
- Hingga mendekati 0 kali.

Total seluruh iterasi adalah:

$$(N - 3) + (N - 4) + (N - 5) + \dots + 1$$

Deret tersebut dapat ditulis sebagai:

$$\frac{(N - 3)(N - 2)}{2}$$

Jika dikembangkan:

$$\frac{N^2 - 5N + 6}{2}$$

Pada analisis Big-O, konstanta dan suku berderajat lebih rendah diabaikan, sehingga diperoleh:

↓  
 $O(N^2)$

2. Perhatikan potongan program berikut:

```
for (int i = 0; i < N; i++) {  
    if (i * i > N) {  
        break;  
    }  
}
```

Kompleksitas potongan program tersebut paling mendekati....

- ☐ a  $O(\log N)$
- ☒ b  $O(\sqrt{N})$
- ☐ c  $O(N/2)$
- ☐ d  $O(N)$

«» C++



```
for (int i = 0; i < N; i++) {  
    if (i * i > N) {  
        break;  
    }  
}
```

### 1. Analisis Perulangan

Perulangan dimulai dari:

«» C++



```
i = 0
```

Kemudian nilai `i` terus bertambah satu per satu.

Namun, perulangan tidak berjalan sampai `i = N`, karena terdapat perintah:

«» C++



```
if (i * i > N) {  
    break;  
}
```

Artinya, perulangan akan berhenti ketika:

$$i^2 > N$$

## 2. Menentukan Banyaknya Iterasi

Dari syarat:

$$i^2 > N$$

Maka nilai  $i$  yang menyebabkan perulangan berhenti adalah sekitar:

$$i \approx \sqrt{N}$$

Sehingga jumlah iterasi yang dilakukan hanya sekitar  $\sqrt{N}$  kali.

## 3. Contoh

Misalkan:

$N = 100$

Nilai  $i$ :

| $i$ | $i^2$            |
|-----|------------------|
| 0   | 0                |
| 1   | 1                |
| 2   | 4                |
| 3   | 9                |
| 4   | 16               |
| 5   | 25               |
| 6   | 36               |
| 7   | 49               |
| 8   | 64               |
| 9   | 81               |
| 10  | 100              |
| 11  | 121 ✖ (berhenti) |

Perulangan hanya berjalan sekitar 11 kali, bukan 100 kali.

#### 4. Analisis Big-O

Karena jumlah iterasi bertambah sebanding dengan akar dari  $N$ , maka kompleksitas waktunya adalah:

$$O(\sqrt{N})$$

---

#### Kesimpulan

✅ Jawaban yang benar adalah:

b.  $O(\sqrt{N})$

Alasan: Perulangan berhenti ketika  $i^2 > N$ , sehingga jumlah iterasi hanya sekitar  $\sqrt{N}$  kali, bukan  $N$  kali.

3. Perhatikan potongan program berikut:

```
counter = N;
while (counter > 0) {
    counter = counter / 3;
}
```

Kompleksitas potongan program tersebut paling mendekati....

- ☒ a  $O(\log N)$
- ☐ b  $O(\sqrt{N})$
- ☐ c  $O(N/3)$
- ☐ d  $O(N)$

## Pembahasan

Perhatikan potongan program berikut:

```
</> C++
counter = N;

while (counter > 0) {
    counter = counter / 3;
}
```

### 1. Analisis Perulangan

Pada awal program:

```
</> C++
counter = N;
```

Kemudian selama nilai `counter` masih lebih besar dari 0, program akan menjalankan:

```
</> C++
counter = counter / 3;
```

Artinya, setiap satu kali perulangan nilai `counter` selalu dibagi 3.

## 2. Contoh Perhitungan

Misalkan:

**N = 81**

| Iterasi | Nilai <code>counter</code> |
|---------|----------------------------|
| 0       | 81                         |
| 1       | 27                         |
| 2       | 9                          |
| 3       | 3                          |
| 4       | 1                          |
| 5       | 0 (berhenti)               |

Terlihat bahwa jumlah iterasi hanya sekitar 5 kali, bukan 81 kali.

### 3. Analisis Matematis

Setelah  $k$  kali perulangan, nilai `counter` menjadi:

$$\frac{N}{3^k}$$

Perulangan berhenti ketika:

$$\frac{N}{3^k} \leq 1$$

Sehingga:

$$3^k \geq N$$

Dengan mengambil logaritma kedua ruas diperoleh:

$$k = \log_3 N$$

Dalam analisis Big-O, basis logaritma tidak memengaruhi kompleksitas karena hanya berbeda faktor konstanta.

Maka kompleksitas waktunya adalah:

$$O(\log N)$$

---

### Kesimpulan

✅ Jawaban yang benar adalah:

a.  $O(\log N)$

Alasan: Pada setiap iterasi, nilai `counter` dibagi 3 sehingga jumlah perulangannya bertambah secara logaritmik terhadap nilai  $N$ .



4. Perhatikan potongan program berikut:

```
for (int i = 0; i < N; i++) {  
    counter = i;  
    while (counter > 0) {  
        counter = counter / 2;  
    }  
}
```

Kompleksitas potongan program tersebut paling mendekati....

- ☒ a  $O(N \log N)$
- ☐ b  $O(\sqrt{N})$
- ☐ c  $O(N^2/2)$
- ☐ d  $O(N^2)$

## Pembahasan

Perhatikan potongan program berikut:

```
<> C++  
  
for (int i = 0; i < N; i++) {  
  
    counter = i;  
  
    while (counter > 0) {  
        counter = counter / 2;  
    }  
  
}
```

### 1. Analisis Perulangan Luar

```
<> C++  
  
for (int i = 0; i < N; i++)
```

Perulangan luar berjalan sebanyak  $N$  kali.

Kompleksitas:

$O(N)$

## 2. Analisis Perulangan Dalam

```
// C++
counter = 1;

while (counter > 0) {
    counter = counter / 2;
}
```

Pada setiap iterasi, nilai `counter` selalu dibagi 2.

Misalnya jika:

```
counter = 16
```

Maka prosesnya:

| Iterasi | Nilai <code>counter</code> |
|---------|----------------------------|
| 0       | 16                         |
| 1       | 8                          |
| 2       | 4                          |
| 3       | 2                          |
| 4       | 1                          |
| 5       | 0 (berhenti)               |

Terlihat bahwa jumlah iterasi tidak sebanyak `counter`, melainkan hanya sekitar:

$$\log_2(\text{counter})$$

Karena nilai terbesar `counter` adalah  $N$ , maka kompleksitas perulangan `while` adalah:

$$O(\log N)$$

### 3. Menggabungkan Kompleksitas

Perulangan luar berjalan:

$$O(N)$$

Di dalamnya terdapat perulangan `while` dengan kompleksitas:

$$O(\log N)$$

Karena keduanya bersarang (nested loop), kompleksitas total diperoleh dengan mengalikan keduanya:

$$O(N) \times O(\log N)$$

Sehingga:

$$O(N \log N)$$

---

### Kesimpulan

✅ Jawaban yang benar adalah:

a.  $O(N \log N)$

Alasan: Perulangan `for` dijalankan sebanyak  $N$  kali, sedangkan setiap iterasi menjalankan `while` yang membagi nilai `counter` menjadi setengah hingga habis, sehingga membutuhkan sekitar  $\log N$  iterasi. Oleh karena itu, kompleksitas total program adalah  $O(N \log N)$ .

5. Perhatikan potongan program berikut:

```
for (int i = 0; i < N; i++) {  
    counter = i;  
    while (counter > 0) {  
        counter = counter / 2;  
    }  
  
    counter = 0;  
    while (counter * counter < N) {  
        counter = counter + 1;  
    }  
}
```

Kompleksitas potongan program tersebut paling mendekati ....

- ☐ a  $O(N \log N)$
- ☒ b  $O(N\sqrt{N})$
- ☐ c  $O(N \log N \sqrt{N})$
- ☐ d  $O(N^2)$

## Pembahasan

Perhatikan potongan program berikut:

```
<< C++  
  
for (int i = 0; i < N; i++) {  
  
    counter = i;  
  
    while (counter > 0) {  
        counter = counter / 2;  
    }  
  
    counter = 0;  
  
    while (counter * counter < N) {  
        counter = counter + 1;  
    }  
}
```

## 1. Analisis Perulangan Luar

```
// C++  
for (int i = 0; i < N; i++)
```

Perulangan luar dijalankan sebanyak  $N$  kali.

Kompleksitas:

$$O(N)$$

## 2. Analisis While Pertama

```
// C++  
counter = i;  
  
while (counter > 0) {  
    counter = counter / 2;  
}
```

Pada setiap iterasi, nilai `counter` dibagi 2.

Contoh:

```
16 → 8 → 4 → 2 → 1 → 0
```

Jumlah iterasinya sekitar:

$$O(\log N)$$

## 3. Analisis While Kedua

```
// C++  
counter = 0;  
  
while (counter * counter < N) {  
    counter = counter + 1;  
}
```

Perulangan berhenti ketika:

$$counter^2 \geq N$$

Sehingga nilai `counter` hanya bertambah hingga sekitar:

$$\sqrt{N}$$

Kompleksitasnya adalah:

$$O(\sqrt{N})$$

#### 4. Menggabungkan Kompleksitas

Di dalam satu kali perulangan `for`, terdapat dua proses:

- While pertama  $\rightarrow O(\log N)$
- While kedua  $\rightarrow O(\sqrt{N})$

Karena keduanya dijalankan secara berurutan, maka kompleksitas per iterasi adalah:

$$O(\log N + \sqrt{N})$$

Untuk nilai `N` yang besar:

$$\sqrt{N} > \log N$$

Sehingga kompleksitas dominannya menjadi:

$$O(\sqrt{N})$$

Karena perulangan luar berjalan sebanyak `N` kali, maka:

$$O(N) \times O(\sqrt{N}) = \boxed{O(N\sqrt{N})}$$

#### 4. Menggabungkan Kompleksitas

Di dalam satu kali perulangan `for`, terdapat dua proses:

- While pertama  $\rightarrow O(\log N)$
- While kedua  $\rightarrow O(\sqrt{N})$

Karena keduanya dijalankan secara berurutan, maka kompleksitas per iterasi adalah:

$$O(\log N + \sqrt{N})$$

Untuk nilai `N` yang besar:

$$\sqrt{N} > \log N$$

Sehingga kompleksitas dominannya menjadi:

$$O(\sqrt{N})$$

Karena perulangan luar berjalan sebanyak `N` kali, maka:

$$O(N) \times O(\sqrt{N}) = \boxed{O(N\sqrt{N})}$$

6. Perhatikan potongan program berikut:

```
total = 0;
for (int i = 0; i < N; i++) {
    for (int j = 1; j < M; j++) {
        total = total + 1;
    }

    for (int j = 0; j < N; j++) {
        total = total + 1;
    }
}
```

Kompleksitas potongan program tersebut paling mendekati....

- ☐ a  $O(N^2)$
- ☐ b  $O(NM)$
- ☒ c  $O(N^2 + NM)$
- ☐ d  $O(N + M)$

## Pembahasan

Perhatikan potongan program berikut:

```
// C++
total = 0;

for (int i = 0; i < N; i++) {

    for (int j = 1; j < M; j++) {
        total = total + 1;
    }

    for (int j = 0; j < N; j++) {
        total = total + 1;
    }

}
```

### 1. Analisis Perulangan Luar

```
// C++
for (int i = 0; i < N; i++)
```

Perulangan luar dijalankan sebanyak  $N$  kali.

Kompleksitas:

$$O(N)$$

### 2. Analisis Perulangan Dalam Pertama

```
// C++
for (int j = 1; j < M; j++)
```

Perulangan ini berjalan sebanyak  $M - 1$  kali.

Dalam analisis Big-O, konstanta tidak diperhitungkan, sehingga:

$$O(M)$$

Karena berada di dalam perulangan luar, maka kompleksitasnya menjadi:

$$O(N \times M) = O(NM)$$



### 3. Analisis Perulangan Dalam Kedua

```
// C++  
for (int j = 0; j < N; j++)
```

Perulangan ini berjalan sebanyak  $N$  kali.

Karena juga berada di dalam perulangan luar, kompleksitasnya adalah:

$$O(N \times N) = O(N^2)$$

### 4. Menggabungkan Kompleksitas

Di dalam satu kali perulangan `for` terdapat dua perulangan yang dijalankan secara berurutan, sehingga kompleksitasnya dijumlahkan:

$$O(NM) + O(N^2)$$

atau dapat ditulis sebagai:

$$O(N^2 + NM)$$

### Contoh Ilustrasi

Misalkan:

- $N = 5$
- $M = 3$

Perulangan pertama:

$5 \times 3 = 15$  iterasi



Perulangan kedua:

$5 \times 5 = 25$  iterasi



Total iterasi:

$15 + 25 = 40$  iterasi



Sesuai dengan:

$$O(NM + N^2)$$

## Kesimpulan

✅ Jawaban yang benar adalah:

c.  $O(N^2 + NM)$

### Ringkasan Analisis

| Bagian Program                                                   | Kompleksitas  |
|------------------------------------------------------------------|---------------|
| Perulangan luar <code>for (i = 0; i &lt; N; i++)</code>          | $O(N)$        |
| Perulangan dalam pertama <code>for (j = 1; j &lt; M; j++)</code> | $O(M)$        |
| Gabungan pertama                                                 | $O(NM)$       |
| Perulangan dalam kedua <code>for (j = 0; j &lt; N; j++)</code>   | $O(N)$        |
| Gabungan kedua                                                   | $O(N^2)$      |
| Kompleksitas total                                               | $O(N^2 + NM)$ |

7. Perhatikan potongan program berikut:

```
total = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        total = total + 1;
    }
}
```

Jika misalkan untuk  $N = 10$  potongan program tersebut membutuhkan waktu 1 detik untuk dijalankan, maka untuk  $N = 100$  potongan program tersebut kira-kira membutuhkan waktu....

- ☐ a 1 detik
- ☐ b 10 detik
- ☒ c 100 detik
- ☐ d 10.000 detik

## Pembahasan

Perhatikan potongan program berikut:

```
</ C++
total = 0;

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        total = total + 1;
    }
}
```

## 1. Analisis Kompleksitas

Perulangan luar:

```
</> C++  
for (int i = 0; i < N; i++)
```

berjalan sebanyak N kali.

Di dalamnya terdapat perulangan:

```
</> C++  
for (int j = 0; j < N; j++)
```

yang juga berjalan sebanyak N kali.

Karena kedua perulangan bersarang (nested loop), maka jumlah total iterasi adalah:

$$N \times N = N^2$$

Sehingga kompleksitas waktunya adalah:

$$O(N^2)$$

## 2. Menghitung Perbandingan Waktu

Diketahui:

- Saat  $N = 10$ , waktu eksekusi = 1 detik.

Jumlah operasi saat  $N = 10$ :

$$10^2 = 100$$

Ketika nilai  $N = 100$ :

$$100^2 = 10.000$$

Perbandingan jumlah operasi:

$$\frac{100^2}{10^2} = \frac{10.000}{100} = 100$$

Artinya, jumlah operasi menjadi 100 kali lebih banyak.

Jika sebelumnya membutuhkan 1 detik, maka waktu yang dibutuhkan menjadi:

$$1 \times 100 = 100 \text{ detik}$$

---

### Contoh Perbandingan

| Nilai N | Jumlah Operasi   | Waktu     |
|---------|------------------|-----------|
| 10      | $10^2 = 100$     | 1 detik   |
| 100     | $100^2 = 10.000$ | 100 detik |

## Kesimpulan

✅ Jawaban yang benar adalah:

c. 100 detik

Alasan: Kompleksitas program adalah  $O(N^2)$ . Ketika nilai  $N$  dinaikkan dari 10 menjadi 100 (10 kali lebih besar), jumlah operasi meningkat menjadi 100 kali lipat, sehingga waktu eksekusi juga diperkirakan menjadi 100 detik.